

PAOS AI_Workflow — Benchmark Audit Results

Raw Score: 148/220 | **Normalized:** 67/100 | **Grade:** C **Auto-grade penalty:** 2 ★ questions failed (Q56, Q81)

Category Breakdown

#	Category	Raw	Max	%
1. OOP	33	36	92%	
2. Data Structures	12	18	67%	
3. Graph Theory	11	12	92%	
4. Linear Algebra	6	8	75%	
5. Digital Logic	8	8	100%	
6. Numerical Analysis	7	8	88%	
7. System Analysis	14	14	100%	
8. Database	10	16	62%	
9. Calculus	4	10	40%	
10. UX/UI	20	20	100%	
11. Security	10	30	33%	
12. API Endpoints	17	20	85%	
13. Code Quality	14	20	70%	
TOTAL	148	220	67%	

Weight Tiers

Tier	Categories	Raw	Max	%
Heavy (48%)	OOP, Data Structures, System Analysis, Digital Logic, Security	77	106	73%
Medium (25%)	Code Quality, API Endpoints, Database	41	56	73%
Moderate (18%)	UX/UI, Linear Algebra	26	28	93%
		22	30	73%

Tier	Categories	Raw	Max	%
Supporting (9%)	Graph Theory, Calculus, Numerical Analysis			

Detailed Results

★ Q1: 2/2

Canvas.tsx uses useNodesState/useEdgesState hooks — internal state hidden behind onSave callback

★ Q2: 2/2

No filesystem paths in API responses

Q3: 1/2

onUpdate dispatched but no input validation before dispatch

Q4: 1/2

Props are typed interfaces. Canvas receives 8+ props

★ Q5: 2/2

Frontend never constructs filesystem paths

Q6: 2/2

Shared utilities in lib/

Q7: 2/2

No class inheritance — all composition via hooks+props

Q8: 2/2

nodeTypes registry pattern

★ Q9: 2/2

AVAILABLE_AGENTS array — all agents share same shape

★ **Q10: 2/2**

execute-flow never checks agentId — polymorphic execution

Q11: 2/2

Adding to AVAILABLE_AGENTS is sufficient

Q12: 2/2

Multiple interfaces — each a coherent concept

Q13: 2/2

Route files, resource-named

Q14: 2/2

Each file has clear purpose

Q15: 2/2

Template system with categories and presets

Q16: 2/2

ReadOnlyNode replaces AgentNode — same data interface

Q17: 1/2

FlowBuilderProps has 8 fields — could be split

Q18: 2/2

Canvas depends on onSave(layout) abstraction

★ **Q19: 2/2**

DAG stored as nodes[] + edges[] — proper graph structure

★ **Q20: 2/2**

Kahn's algorithm (inDegree + shift) — correct BFS topological sort

×

☑

Q21: 0/2

☑

No cycle detection — Kahn's silently truncates cyclic DAGs

▲

Q22: 2/2

▲

Queue with enqueue/dequeue/done + cleanup on GET

▲

Q23: 1/2

▲

Some .find() calls — acceptable for small arrays

☑

Q24: 1/2

☑

Some .includes() calls — some should be Set.has()

▲

Q25: 2/2

▲

Recursive renderNode — proper tree rendering

×

Q26: 1/2

×

selectedSkills is array but skills unique — should use Set

☑

Q27: 0/2

☑

No DAG validation on read

☑

Q28: 2/2

☑

Branching handled — cascade spawns all ready nodes

☑

Q29: 2/2

☑

Join nodes wait for all predecessors

☑

Q30: 2/2

☑

All FSM states present in ReadOnlyNode

Q31: 2/2

Topological sort is deterministic

▲

☑

Q32: 1/2

☑

Some permission checks use `.filter/.includes`

☑

Q33: 2/2

☑

Same pipeline always produces same execution order

☑

Q34: 2/2

☑

Viewport transforms saved/restored

☑

Q35: 2/2

☑

DAG coordinates preserved across sessions

✗

Q36: 2/2

✗

Consistent node spacing

☑

Q37: 0/2

☑

No similarity metrics — exact-match only

☑

Q38: 2/2

☑

All visibility conditions combinatorial

☑

Q39: 2/2

☑

Cascade is sequential circuit — depends on stored state

☑

Q40: 2/2

☑

Permission checks are pure

☑

Q41: 2/2

☑

`pipeline-flow.json` enables crash recovery

Q42: 2/2

No float equality comparisons

Q43: 2/2

Slider values rounded to nearest step

Q44: 2/2

Progress is monotonic

Q45: 1/2

Basic edge cases handled, no explicit NaN/Infinity guards

★ Q46: 2/2

Frontend uses API exclusively

Q47: 2/2

Clean 4-layer architecture

Q48: 2/2

Moderate coupling

Q49: 2/2

High cohesion

Q50: 2/2

UI never mixed with business logic

Q51: 2/2

Queue fully separate from pipeline data

Q52: 2/2

Execute-flow works without queue

★ Q53: 2/2

Atomic write order: artifacts → flow → META

■

■

★ Q54: 2/2

■

Crash recoverable with fallback defaults

×

Q55: 2/2

×

Schema validation at read time

■

★ Q56: 0/2

■

No caching — same file read multiple times per request

■

Q57: 2/2

■

Directory structure as index

×

Q58: 2/2

×

All JSON.parse calls have try/catch defaults

■

Q59: 0/2

■

No concurrent write prevention

■

★ Q60: 2/2

■

Correct write order verified

×

Q61: 2/2

×

Progress as absolute counter

■

Q62: 0/2

■

No velocity tracking

×

Q63: 2/2

×

Cumulative totals correct

Q64: 0/2

No moving averages

x

✓

Q65: 0/2

✓

No multi-variable optimization

✓

★ Q66: 2/2

✓

2s polling, spinners, progress bars, PID display

✓

★ Q67: 2/2

✓

4 recovery paths: Retry, Skip, Cancel, Intervene

✓

Q68: 2/2

✓

Type confirmation to delete

✓

Q69: 2/2

✓

Responsive at 390px

✓

Q70: 2/2

✓

Responsive at 3840px

✓

Q71: 2/2

✓

Scroll-wheel on all sliders

✓

Q72: 2/2

✓

Specific error messages

✓

Q73: 2/2

✓

Gold primary, dark theme consistent

✓

Q74: 2/2

✓

Progressive disclosure

Q75: 2/2

Affordances clear

■

■

★ Q76: 2/2

■

API keys in .env only

■

★ Q77: 2/2

■

.env in .gitignore

▲

★ Q78: 2/2

▲

spawn() with fixed commands, no unsanitized input

▲

Q79: 1/2

▲

path traversal audit needed for join() calls

×

Q80: 1/2

×

Basic input validation, no comprehensive type checking

×

★ Q81: 0/2

×

No rate limiting — any endpoint can be hammered

■

Q82: 0/2

■

No CORS configuration

▲

Q83: 2/2

▲

No secrets in console.log

×

Q84: 1/2

×

No sandbox for agents, acceptable for dev

■

★ Q85: 0/2

■

No authentication on any endpoint

Q86: 2/2

No dangerouslySetInnerHTML

✖

✔

Q87: 0/2

✔

No CSRF protection

✔

Q88: 2/2

✔

npm audit: 0 critical vulnerabilities

✖

Q89: 2/2

✖

No hardcoded tokens

✔

Q90: 0/2

✔

No HTTPS cookies or HSTS

✔

Q91: 2/2

✔

No duplicate resources

✔

Q92: 2/2

✔

Consistent HTTP methods

▲

Q93: 2/2

▲

Consistent response envelope

✔

Q94: 1/2

✔

Some 500s instead of 404s for missing resources

✖

Q95: 2/2

✖

No redundant CRUD

✔

Q96: 0/2

✔

No pagination — will break >1000 pipelines

Q97: 2/2

kebab-case, plural



Q98: 2/2



No orphaned endpoints



Q99: 2/2



Reasonable surface area



Q100: 2/2



Full CRUD on most resources



Q101: 2/2



TypeScript strict mode



Q102: 0/2



~13 any types found — should be zero



★ Q103: 2/2



All API routes in try/catch



Q104: 1/2



Some .then() chains — prefer async/await



Q105: 2/2



No barrel re-exports



Q106: 1/2



Some PascalCase non-component files



Q107: 2/2



Single source of truth

Q108: 0/2

Hardcoded /home/dev/ paths in routes

■

×

Q109: 2/2

×

Hooks at top level

Q110: 0/2

Components without React.memo causing re-renders

Gaps Found

Gaps are questions scored < 2. Each has an implementation plan in `.hermes/plans/` (user-triggered only).

Gap 1 — Q21

- **Score:** 0/2
- **Severity:** High
- **Plan:** `.hermes/plans/benchmark-gap-1.md`
- **Status:** Pending

Gap 2 — Q27

- **Score:** 0/2
- **Severity:** High
- **Plan:** `.hermes/plans/benchmark-gap-2.md`
- **Status:** Pending

Gap 3 — Q37

- **Score:** 0/2
- **Severity:** Medium
- **Plan:** `.hermes/plans/benchmark-gap-3.md`
- **Status:** Pending

Gap 4 — Q56

- **Score:** 0/2
- **Severity:** Critical ★
- **Plan:** `.hermes/plans/benchmark-gap-4.md`
- **Status:** Pending

Gap 5 — Q59

- **Score:** 0/2
- **Severity:** High
- **Plan:** `.hermes/plans/benchmark-gap-5.md`

- **Status:** Pending

Gap 6 — Q62

- **Score:** 0/2
- **Severity:** Medium
- **Plan:** .hermes/plans/benchmark-gap-6.md
- **Status:** Pending

Gap 7 — Q64

- **Score:** 0/2
- **Severity:** Medium
- **Plan:** .hermes/plans/benchmark-gap-7.md
- **Status:** Pending

Gap 8 — Q65

- **Score:** 0/2
- **Severity:** Medium
- **Plan:** .hermes/plans/benchmark-gap-8.md
- **Status:** Pending

Gap 9 — Q81

- **Score:** 0/2
- **Severity:** Critical ★
- **Plan:** .hermes/plans/benchmark-gap-9.md
- **Status:** Pending

Gap 10 — Q82

- **Score:** 0/2
- **Severity:** High
- **Plan:** .hermes/plans/benchmark-gap-10.md
- **Status:** Pending

Gap 11 — Q85

- **Score:** 0/2
- **Severity:** Critical ★
- **Plan:** .hermes/plans/benchmark-gap-11.md
- **Status:** Pending

Gap 12 — Q87

- **Score:** 0/2

- **Severity:** High
- **Plan:** .hermes/plans/benchmark-gap-12.md
- **Status:** Pending

Gap 13 — Q90

- **Score:** 0/2
- **Severity:** High
- **Plan:** .hermes/plans/benchmark-gap-13.md
- **Status:** Pending

Gap 14 — Q96

- **Score:** 0/2
- **Severity:** High
- **Plan:** .hermes/plans/benchmark-gap-14.md
- **Status:** Pending

Gap 15 — Q102

- **Score:** 0/2
- **Severity:** High
- **Plan:** .hermes/plans/benchmark-gap-15.md
- **Status:** Pending

Gap 16 — Q108

- **Score:** 0/2
- **Severity:** High
- **Plan:** .hermes/plans/benchmark-gap-16.md
- **Status:** Pending

Gap 17 — Q110

- **Score:** 0/2
- **Severity:** High
- **Plan:** .hermes/plans/benchmark-gap-17.md
- **Status:** Pending

Summary

17 gaps found. Top 3 priorities:

1. **Rate limiting (Q81)** — no DOS protection at all. Critical security gap.
2. **No authentication (Q85)** — anyone with network access can delete pipelines.

3. **File caching (Q56)** — same files read multiple times per request.

Audited: 25 June 2026 | Benchmark: Coding-Principles-Benchmark
v2.0